

Мисловни вериги и модели за разсъждение

Лекция 10 — Сесия 2

Преди почивката (Сесия 1)

- Генерирането е итеративно теглене на извадки от вероятностно разпределение
- Контролираме шума с **temperature, top-k, top-p**
- Подканите работят, защото попадат в шаблоните от SFT данните (Лекция 8)
- Видяхме на живо как примерите, ролята и форматът променят изхода

Сега: какво се случва, когато задачата изисква **разсъждение**, а не разпознаване на шаблон.

Какво ще покрием

1. **Мисловни вериги** — как накараме модела да мисли стъпка по стъпка
2. **Как се обучават моделите за разсъждение** — следи, награди, RL
3. **Метрики и компромиси** — къде печелят и какво струват
4. **Кога какво да използваме** — оценяване на подкани и кеширане на подкани

Мисловни вериги (Chain of Thought)

Защо моделът се проваля на многоетапни задачи

Пример: „Иван има 23 ябълки. Дава 7 на Мария, после купува още 12. Колко има сега?“

Подкана: „Дай само числовия отговор.“

Отговор: 35

Моделът трябва да направи всички стъпки в **един проход напред**, без място да задържи междинни резултати. На по-сложни задачи това се чупи.

Решението: дайте му **място да мисли** — токени за междинно разсъждение.

Мисловни вериги (Chain of Thought)

Принуждаваме модела да напише разсъждението си, преди да даде отговор.

Иван има 23 ябълки. Дава 7 на Мария, после купува още 12.
Колко има сега? Помисли стъпка по стъпка.

Започва с 23.

След като дава 7: $23 - 7 = 16$.

След като купува 12: $16 + 12 = 28$.

Отговор: 28.

Междинните токени служат като **работно поле**. Всеки следващ токен е условен от вече написаните.

Мисловните вериги не променят архитектурата — само формата на изхода.

Мисловни вериги без примери

Една изречение, която често е достатъчна (Kojima et al. 2022):

„Помисли стъпка по стъпка.“ („Let's think step by step.“)

Защо работи: SFT данните съдържат много примери, в които правилен отговор се предхожда от разсъждение.

Кога да я опитате първо: проста задача, която моделът се проваля да реши директно.

Мисловни вериги с примери

Показваме **изработени примери** с разсъждение в подканата.

В: Сара има 5 ябълки и купува още 3. Колко има?

О: Започва с 5. След като купува 3: $5 + 3 = 8$. Отговор: 8.

В: Петър има 12 круши и дава 4. Колко му остават?

О: Започва с 12. След като дава 4: $12 - 4 = 8$. Отговор: 8.

В: Иван има 23 ябълки. Дава 7 и купува 12. Колко има?

О:

Wei et al. (2022) — оригиналният труд. Показва на модела **формата** на разсъждението, не само формата на отговора.

Мисловните вериги работят главно при мащаб

[ИЗОБРАЖЕНИЕ: cot-emergent-scale] — диаграма от Wei 2022: точност спрямо размер на модела, със и без мисловни вериги. Линиите се разделят след $\sim 62B$ параметри.

- Под определен размер мисловните вериги **не помагат** (а понякога вредят)
- Над прага дават съществен скок при многоетапни задачи
- Малък модел, който „мисли“, произвежда шум — не разсъждение

Връзка с Лекция 7 (закони за мащабиране и възникващи способности).

Self-consistency — кратко отстъпление

Идеята (Wang et al. 2022):

1. Задавате една и съща подкана с разсъждение K пъти при temperature > 0
2. Получавате K различни вериги на разсъждение
3. Гласувате — мажоритарният финален отговор печели

Полезно, когато отговорите са кратки и дискретни (число, етикет, избор).

Паралелно и статистическо. Скъпо: K пъти повече разходи.

Лимитът на мисловните вериги чрез подкана

Мисловните вериги са поведение, изтръгнато от модел, който не е обучен да разсъждава.

- Подканваме го — но обучителната му цел е била „предвиди следващ токен“
- Веригите са едноточни: моделът не може да се върне и да поправи
- На дълги, многоклонови задачи това се чупи

Какво следва, ако наистина искаме модел, който разсъждава добре?

Тренирайте го за това.

Как се обучават моделите за разсъждение

Не е само „мисловни вериги с друг етикет“

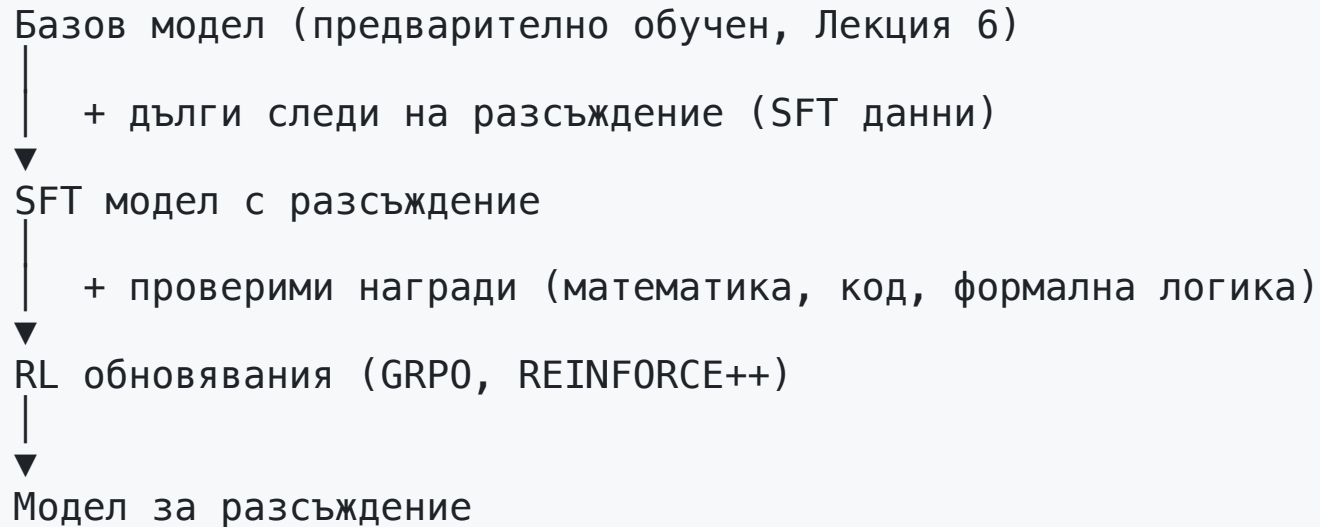
Моделите за разсъждение (o1, R1, Qwen-QwQ) споделят архитектурата с базовите модели.

Разликата е в **post-training**:

Стъпка	Базов / инструкционен модел	Модел за разсъждение
Предварително обучение	предвиждане на следващ токен	същото
SFT	демонстрации (подкана, отговор)	дълги следи на разсъждение
RL	RLHF — научен модел на наградата	RLVR — проверими награди

Архитектурата е същата. Целевата функция при последния етап е друга.

Пайплайнът от един поглед



Двата нови елемента: **следите** като SFT данни и **проверимостта** на наградата.

Откъде идват следите на разсъждение

Следа на разсъждение (thought trace) = вътрешният монолог на модела, преди финалния отговор. Стотици до хиляди токени за един пример.

Източници на SFT следи:

- **Синтетично генериране** от силен модел — пускате го в режим на дълго разсъждение и филтрирате по правилен отговор
- **Дистилация от o1/R1 изходи** — моделите-учители показват веригата
- **Човешки следи** — скъпи, но висок сигнал
- **Смесени корпуси** — math, code, формални докази, логически задачи

Качеството на следите е по-важно от количеството. Лоши следи учат лоши навици.

RLVR — обучение с подсилване с проверими награди

Различно от RLHF (Лекция 8): **няма** научен модел на наградата.

Тип задача	Проверител на наградата
Математика	сравнение с верния отговор
Програмиране	пускане на тестове
Формална логика	формален верификатор
Структуриран изход	проверка по схема

Награда: **+1 ако е вярно, 0 иначе**. Просто, евтино, прецизно — но работи само когато имаме ground truth.

Process supervision срещу outcome supervision

Lightman et al. (2023) — „Let's Verify Step by Step“.

Подход	Какво се оценява	Цена	Резултат
Outcome supervision	само финалният отговор	евтино	работи, но допуска „правдоподобни, но грешни“ вериги
Process supervision	всяка междинна стъпка	по-скъпо (повече етикети)	по-малко грешки в разсъждението

Process supervision дава по-стабилни модели за разсъждение, защото грешките се хващат рано в веригата.

GRPO и DeepSeek-R1

GRPO (Group Relative Policy Optimization) — RL алгоритъм:

- Генерираме K отговора за една подкана
- Оценяваме всеки с проверителя
- Отклонението на наградата спрямо средното за групата дава сигнала
- Не изисква отделен модел на стойността (за разлика от PPO)

DeepSeek-R1 (2025) — отворената референция:

- Същата парадигма като o1
- **Видими** токени на разсъждение
- Обучен с GRPO върху математика и код

Връзка с Лекция 8 (споменахме GRPO в края).

Защо това работи (засега) за разсъждение

RLVR изисква **проверима истина**.

Имаме ли ground truth?	Пример	Работи ли?
Да	$7 \times 8 = ?$	да
Да	програма + тестове	да
Да	формално доказателство	да
Не	„напиши хубаво есе“	не
Не	„кой превод е по-точен“	не

Затова разсъждаващите модели първо успяват в математика и код. Дали и как ще се пренесе към отворени задачи — открит въпрос за изследване.

Метрики и компромиси по компютърно време

Скок в бенчмарковете

Конкретни числа, не „по-добре е“:

Бенчмарк	Стандартен модел	Модел за разсъждение
AIME 2024 (математика)	~13%	~80%+
MATH-500	~75%	~95%+
GPQA Diamond (наука)	~40%	~75%+
Codeforces (рейтинг)	~800	~2000+

[ИЗОБРАЖЕНИЕ: reasoning-benchmarks] — диаграма с двойки колонки за всеки бенчмарк.

Това не е малко подобрение. На AIME е разликата между „случаен резултат“ и „поне олимпийски стипендиант“.

Където моделите за разсъждение НЕ печелят

- Прости фактологически въпроси („Коя е столицата на Франция?“)
- Резюмета и парафрази
- Рутинни редакции на код
- Превод на стандартен текст
- Класификация по ясни критерии

На тези задачи: **същата точност**, но **много по-висока** латентност и цена.

Не посягайте за разсъждаващ модел рефлекторно — посягайте, когато задачата има стъпки, които си струват токени.

Компромис: токени срещу точност

[ИЗОБРАЖЕНИЕ: tokens-vs-accuracy] — крива: токени на разсъждение по оста X, точност по Y. Расте, после плато.

- Трудна математическа задача → 5 000–20 000 токена на разсъждение
- Латентност: 30 секунди до 2 минути на отговор
- Цена: 5–50x повече от стандартно извикване
- Платото е реално — повече токени не носят винаги повече точност

Моделите за разсъждение **харчат компютърно време срещу точност**. Това е новата ос на дизайна.

Test-time compute като нов мащабиращ ос

Лекция 7: законите за мащабиране казваха — повече параметри, повече данни, повече тренировъчно компютърно време → по-добър модел.

Сега: **успоредна крива на изводно компютърно време.**

Стратегия	Компромис
Голям модел + малко мислене	висока цена за тренировка, ниска за извикване
По-малък модел + повече мислене	ниска цена за тренировка, висока за извикване

Snell et al. (2024) — на много задачи може **по-малък модел + дълго разсъждение** да настигне голям модел с директен отговор.

Това променя как избираме модел за продукт.

Демо: стандартен срещу разсъждаващ модел

Ще пуснем три задачи на двата модела успоредно:

1. **Логическа загадка** — очакваме разсъждаващият да печели категорично
2. **Многоетапна математика** — гледаме веригата, не само отговора
3. **Тривиален фактологически въпрос** — и двата са верни, но разсъждаващият е 20x по-бавен

Целта: да видите кога **компромисът** си струва и кога не.

↓ Тетрадка: Демо

Демото продължава в тетрадката

Кога какво да използваме

Решетка за решение

Тип задача	Първи опит	Ако не стига
Прост факт, превод, парафраза	директна подкана	+ примери
Класификация, извличане	директна подкана + JSON схема	few-shot
Многоетапна аритметика, логика	мисловни вериги	self-consistency
Сложен математически/научен проблем	модел за разсъждение	—
Дълги процедурни задачи (агенти)	мисловни вериги + chaining (Лекция 12)	модел за разсъждение

Започвайте с най-евтиното, което работи. Ескалирайте при нужда.

Оценяване на подкани (evals)

Без оценяване, инженерството на подканите е „интуиция“. Подканите трябва да се третират като код — с регресионни тестове.

Метод	Кога е подходящ	Уловка
Златен набор + assertions	детерминистични задачи	нужно е ground truth
LLM-as-judge	отворени отговори	предубеждения (дължина, ред, self-preference)
A/B с хора	продуктови решения	бавно, скъпо, но реално

Инструменти: `promptfoo`, Anthropic evals, OpenAI evals, `inspect-ai`.

Кеширане на подкани (prompt caching)

Дълги стабилни префикси (системна подкана, few-shot примери) могат да се кешират от страна на сървъра.

Доставчик	Праг	TTL
Anthropic	≥1024 токена	~5 мин
OpenAI	автоматично за повторения	променлив

Ефекти:

- 50–90% спестяване на разходи на повторни извиквания
- Намалена латентност на първия токен

Структурирани подкани със стабилни префикси са едновременно **оценени** и **кешируеми**. Двете причини да пишете подканите като код.

Бърз тест

За всяка задача — кой подход бихте опитали първо?

1. Превод на английска статия на български
2. Намиране на оптимален път в мрежа от 20 възела с теглата
3. Извличане на име, дата и сума от фактура като JSON
4. Доказателство, че $\sqrt{2}$ е ирационално
5. Кратко резюме на чат с клиент за вътрешен тикет

(Обсъждаме всяка след 1 минута.)

Дискусия — можем ли да вярваме на веригата?

Когато моделът напише разсъждение и стигне до отговор:

- **Винаги ли отговорът следва от веригата?** Не. Понякога моделът стига до правилен отговор по неправилни причини — или обратното.
- **Защо изобщо моделът пише точно тази верига?** Защото е статистически вероятна, не задължително защото е валиден разсъждение.

Това е **проблемът на достоверността на разсъждението** (faithfulness). Открит въпрос — особено за приложения с висока цена на грешка.

Накратко

- **Мисловни вериги** са инженерен трик — изтръгват разсъждение от модел, който не е специално обучен за него
- **Моделите за разсъждение** са следваща стъпка — обучени с RL върху проверими награди
- Печелят съществено на математика, код, наука. На рутинни задачи са скъпи без полза.
- **Test-time compute** е нова ос на компромиса
- **Оценявайте** подканите си. **Кеширайте** стабилните префикси.
- Не вярвайте сляпо на веригата — отговорът и разсъждението не са винаги едно цяло.

Какво ни чака в Лекция 11

Подканите и разсъждението предполагат, че **моделът знае отговора** (или може да го извлече от това, което знае).

Какво се случва, когато:

- Не знае?
- Знае, но самоуверено бърка?
- Знаенето трябва да се **обнови** след тренировката?

Халюцинации и RAG — следваща седмица.